ACCOUNTANT HUB

ARCHITECTURE & TECHNICAL DESIGN REPORT

Full-Stack Solo Build · Domain-Driven Design

Mohamed Hossam Hassan
mohamed.hhossam@alum.bue.edu.eg · +201028992674

A comprehensive architecture & technical design report for the Accountant Hub project — a domain-specific accounting marketplace built as a full-stack solo build.

17 Database Tables	**77** API Endpoints	**4** User Roles
15+ UI Components	**30%** Time on Architecture	**10+** Extra Features

Built architecture-first: database, API contracts, component tree, and business-logic flows were fully designed before a single line of code was written.

COVER LETTER

Dear Hiring Team,

I am submitting Accountant Hub as my response to the full-stack assessment. What you will find in this document — and in the codebase — is not a minimum-viable pass at the requirements. It is a deliberate, domain-informed product built the way I believe software should be built: architecture before code, domain knowledge before features, and quality before speed.

I spent approximately 30% of the total project time on pure design before writing a single line of code. Every table, every endpoint, every component was mapped out with the accounting domain in mind. The result is a system that reflects how accounting engagements actually work — not a generic job board with accounting labels applied.

The platform covers all required features, all stated bonus points, and goes significantly further: a full administration role, a client role with complete job lifecycle management, a 5-dimension match scoring engine, NDA gates, certification expiry tracking, structured bid proposals with milestones and engagement terms, an immutable audit trail, and full EN/AR internationalisation with RTL support.

I chose to deviate from the suggested Laravel + MySQL stack. PostgreSQL was necessary for JSONB with GIN indexing — the accounting domain's flexible certification and jurisdiction data cannot be served performantly by MySQL's JSON implementation. FastAPI was chosen for native async and auto-generated OpenAPI documentation. These are engineering decisions, not preferences, and each is documented with rationale in Section 2.

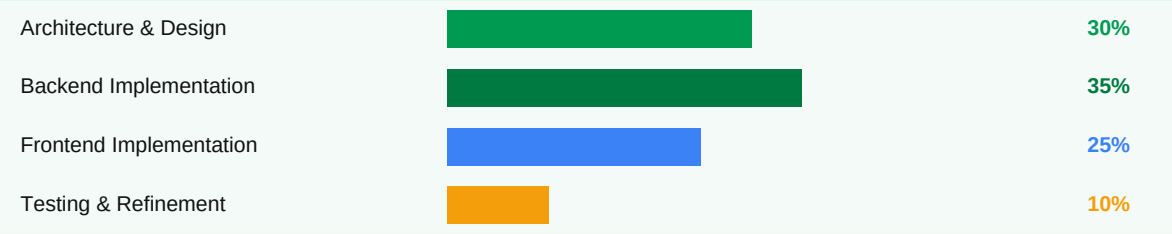
I look forward to walking you through the system.

Mohamed Hossam Hassan
mohamed.hhossam@alum.bue.edu.eg · +201028992674

TABLE OF CONTENTS

1 ARCHITECTURE-FIRST DESIGN PHILOSOPHY

A deliberate, disciplined decision was made to invest approximately **30% of the total project time purely in architecture and design** before a single line of code was written. This reflects a professional engineering mindset — the kind that separates maintainable, scalable systems from ones that accumulate technical debt from the first commit.



What was architected before coding began:

- **Database Schema:** 17 tables with all relationships, constraints, indexes, and JSONB structures fully mapped
- **API Contract:** 77 endpoints across 4 user contexts with request/response shapes defined upfront
- **Component Tree:** Full React component hierarchy and routing structure documented
- **Business Logic Flows:** NDA gates, match scoring, bid acceptance, job closure — all flow-charted
- **Domain Modeling:** Certifications, jurisdictions, accounting standards, and structured bids designed specifically for the accounting vertical

Every implementation decision had a documented rationale, no schema migrations were needed mid-build, and the API surface was coherent from day one. It also enabled **domain-informed design** — the architecture reflects how accounting work actually operates, not a generic job-board template.

2 TECHNOLOGY STACK & RATIONALE

The task brief suggested a Laravel + MySQL stack. After architectural analysis of the domain requirements, an alternative stack was selected with documented rationale for every deviation.

Layer	Technology	Why Chosen
Frontend	React 18 (Vite, JSX)	Component-based architecture, fast HMR, rich ecosystem
Backend	FastAPI (Python 3.10+)	Native async, auto-generated Swagger docs, Pydantic validation — superior for domain-specifi
Database	PostgreSQL	JSONB + GIN indexes for cert/software/jurisdiction fields; CHECK constraints — not achievable
ORM	SQLAlchemy 2.0 (async)	Mature async support, type safety
Auth	JWT (python-jose + bcrypt)	Stateless, FastAPI standard, tab-isolated via sessionStorage
State Mgmt	React Context + TanStack Query v5	Context for auth; Query for server cache, pagination, auto-refetch
Styling	CSS Variables	Full theming control, no framework overhead, dark/light support
i18n	i18next + react-i18next	EN/AR with RTL support, Cairo font for Arabic

Stack Deviation Rationale: PostgreSQL was chosen over MySQL because the accounting domain requires flexible JSON fields (certifications, software skills, jurisdictions) with GIN indexing for performant filtering — a feature MySQL's JSON implementation does not match. FastAPI was chosen over Laravel for native async support and automatic OpenAPI documentation generation.

3 DATABASE DESIGN — 17 TABLES

The schema was designed to serve the accounting domain specifically — not as a generic job-board. Every table, constraint, and index reflects deliberate domain-driven design.

Core Entities

#	Table	Purpose	Key Design Decisions
1	users	All roles: admin, client, accountant	role ENUM, restrictions JSONB, audit_enabled flag
2	client_profiles	Company details	company_name, industry, size, country FK
3	accountant_profiles	Professional details	JSONB: jurisdictions_served, accounting_standards
4	jobs	Job postings	25+ accounting-specific fields, JSONB arrays, denormalized bid counters
5	bids	Structured proposals	UNIQUE(job_id, accountant_id), services_included JSONB, milestones JSONB
6	categories	Hierarchical job categories	Self-referencing parent_id, 24 seeded categories

Reference, Pivot & Supporting Tables

#	Table	Purpose / Contents
7	certifications	CPA, CMA, ACCA, EA, CFA, CIA, DipIFRS, SOCPA (8 predefined)
8	software_skills	QuickBooks, Xero, SAP, Oracle NetSuite + 10 others (14 seeded)
9	countries	10 countries with currency codes and phone validation
10	platform_content	DB-driven disclaimers, NDA text, reminders (9 content keys)
11	accountant_certifications	Pivot: Accountant ↔ Certifications (custom allowed, expiry dates)
12	accountant_software_skills	Pivot: Accountant ↔ Software Skills
13	nda_acceptances	NDA acceptance log — UNIQUE per user per job, IP + timestamp
14	job_attachments	Files attached to job postings
15	uploaded_documents	User PDFs/CVs with admin verification workflow
16	audit_logs	Immutable action trail — WHO, WHAT, WHEN, before/after JSONB
17	password_reset_tokens	Secure password reset functionality

Key Schema Design Features

✓ JSONB columns with GIN indexes for flexible, queryable skill/certification/jurisdiction data

✓ CHECK constraints for data integrity (budget_max >= budget_min, certification_id OR custom_certification_name)

✓ Denormalized counters (bids_count, bids_min_price, bids_max_price) for high-performance job listing queries

✓ Auto-update triggers on 12 tables for updated_at timestamps — no application-layer management needed

✓ Full audit fields (created_at, updated_at, created_by, updated_by) on all main tables

✓ UNIQUE constraint on (job_id, accountant_id) in bids table — duplicate prevention at the database level

Entity Relationship Overview

```
users ■■■■■■■■■■■■■■■■ client_profiles (1:1)
users ■■■■■■■■■■■■■■■■ accountant_profiles (1:1)
users ■■■■■■■■■■■■■■■■ jobs (1:M as client)
users ■■■■■■■■■■■■■■■■ bids (1:M as accountant)
users ■■■■■■■■■■■■■■■■ nda_acceptances (1:M)
users ■■■■■■■■■■■■■■■■ audit_logs (1:M)
accountant_profiles ■ accountant_certifications (1:M)
certifications ■■■■■■■■ accountant_certifications (1:M)
categories ■■■■■■■■■■■■ categories (self-referencing)
categories ■■■■■■■■■■■■ jobs (1:M)
jobs ■■■■■■■■■■■■■■■■■■■■ bids (1:M)
jobs ■■■■■■■■■■■■■■■■■■■■ nda_acceptances (1:M)
```

4 DOMAIN-SPECIFIC ACCOUNTING FEATURES

This system was not built as a generic freelance marketplace with accounting labels applied. Every feature reflects how accounting engagements actually work in practice. The architecture considered the domain it serves at every design decision.

4.1 Certification Tracking

Eight predefined certifications (CPA, CMA, ACCA, EA, CFA, CIA, DipIFRS, SOCPA) with custom certification support via CHECK constraint. Expiry date tracking with 30-day advance warning and automatic exclusion from match scores. The system applies soft matching: missing or expired certifications generate warnings but never block bid submission — the market decides suitability.

4.2 Jurisdiction & Accounting Standards

Jobs specify jurisdiction (US, UK, UAE, KSA, QA, etc.) and accounting standard (GAAP, IFRS, Local GAAP). Accountants declare the jurisdictions they serve and standards they know. The match engine cross-references both dimensions and surfaces mismatches clearly.

4.3 Hierarchical Job Categories

Self-referencing category tree with 6 parent and 18 child categories covering the full accounting taxonomy: Tax Services, Audit & Assurance, Bookkeeping, Financial Advisory, Forensic Accounting, and Specialised (IFRS Conversion, ESG Reporting, Crypto Accounting).

4.4 Structured Bid Proposals

Bids are not generic free-text boxes. Every bid includes: pricing model (fixed/hourly/retainer), all-fees-included transparency flag, services included checklist, milestone-based delivery timeline, engagement terms (mini engagement letter), number of entities, estimated hours, and jurisdiction confirmation. This reflects real accounting proposal practice.

4.5 NDA Gate Flow

Per-job NDA flag with NDA text served from the database (not hardcoded, allowing admin updates). Acceptance is logged with timestamp and IP address. One acceptance per user per job enforced at the database level (UNIQUE constraint). Only accountants can accept; clients see an informational message.

4.6 Soft Match Scoring Engine

A 5-dimension match engine cross-references the accountant's full profile against job requirements: certifications (with expiry awareness), software skills, jurisdictions, accounting standards, and years of experience. Returns a percentage score with matched items in green, missing items in amber, and explicit warnings for expired certifications. Submission is never blocked — the score is advisory.

5 API ARCHITECTURE — 77 ENDPOINTS

All endpoints follow RESTful conventions under **/api/v1/** versioning with a standard response envelope: **{data, meta: {page, per_page, total}}**. JWT Bearer authentication is applied to all protected routes via a **require_role()** dependency. Swagger documentation is auto-generated at **/docs**.

Public (8)

```
POST /api/v1/auth/register & POST /api/v1/auth/login
GET /api/v1/jobs (10 filter params + sort + pagination)
GET /api/v1/jobs/{id} (includes current_user_bid if authenticated)
GET /api/v1/categories, /certifications, /countries, /content/{key}
```

Accountant (11)

```
POST /api/v1/jobs/{id}/accept-nda (logged with IP + timestamp)
POST /api/v1/jobs/{id}/bids (structured proposal, duplicate-checked)
GET /api/v1/jobs/{id}/match-score (5-dimension profile analysis)
GET/PUT /api/v1/my-profile (4 sub-resources: profile, certs, skills, docs)
GET /api/v1/my-bids (all bids with status tracking)
POST/GET/DELETE /api/v1/my-profile/documents
```

Client (9)

```
POST/GET/PUT /api/v1/my-jobs (full job lifecycle management)
PATCH /api/v1/my-jobs/{id}/status (manual open/close)
GET /api/v1/my-jobs/{id}/bids (with accountant display names)
PATCH /api/v1/my-jobs/{id}/bids/{bid_id}/accept (closes job, rejects others)
GET /api/v1/accountants/{id}/profile?job_id=X (only if bid on client's job)
```

Admin (49)

```
Full CRUD for: users, jobs, bids, categories, certifications, software-skills, countries,
documents, content
GET /api/v1/admin/dashboard/stats (breakdown by role, job status, bid status)
PATCH /api/v1/admin/me/audit-toggle (individual admin audit control)
PATCH /api/v1/admin/users/{id}/restrictions (per-user JSONB permissions)
GET /api/v1/admin/audit-logs (filtered by category, role, date, search)
```

Job Filter Parameters (10 dimensions)

Parameter	Description
?search=term	Full-text search on title + description
?category=slug	Filter by category slug
?budget_min=X / ?budget_max=Y	Contained budget range filter
?currency=USD	Currency filter
?pricing_model=fixed	Fixed / hourly / retainer

?jurisdiction=US	Jurisdiction filter
?accounting_standard=GAAP	Accounting standard filter
?status=open	Open / closed / empty (all)
?sort=newest	newest / budget_high / budget_low / bids / deadline
?page=1&per_page=12	Offset pagination

6 FRONTEND ARCHITECTURE

Component Tree

```
App
■■■ Layout (Navbar, Footer)
■■■ AuthProvider (React Context)
■■■ Public Pages
■ ■■■ JobListPage (hero, search, sidebar filters, job cards, pagination)
■ ■■■ JobDetailPage (NDA gate → full details → match score → bid form modal)
■ ■■■ LoginPage, RegisterPage, TermsPage, PrivacyPage
■■■ Accountant Pages
■ ■■■ Dashboard (stats, recent bids)
■ ■■■ MyBidsPage (all bids with status badges)
■ ■■■ ProfilePage (4 tabs: Profile, Certifications, Software Skills, Documents)
■■■ Client Pages
■ ■■■ Dashboard (stats, posted jobs)
■ ■■■ PostJobPage / EditJobPage (full form with validation)
■ ■■■ ClientJobDetailPage (bids list with accept/reject)
■ ■■■ AccountantProfileView (bidder profile — access-gated by bid relationship)
■■■ Admin Pages
■■■ Dashboard (stats + audit toggle)
■■■ UsersPage, JobsPage, BidsPage
■■■ CategoriesPage, CertificationsPage, SoftwareSkillsPage
■■■ CountriesPage, DocumentsPage, ContentPage
■■■ AuditLogsPage (filterable by category, role, date, search)
```

Design System

Token	Value / Decision
Primary colour	#019A51 (as specified in brief)
Inspiration	Designed with Upwork-style UX patterns in mind — clean job cards, sidebar filters, bid flow — adapted and domain-s
Typography (EN)	Playfair Display (headings) + DM Sans (body)
Typography (AR)	Cairo — full RTL layout support
Spacing unit	4px base grid
Breakpoints	Mobile 320px+, Tablet 768px+, Desktop 1024px+
Token storage	sessionStorage — tab-isolated, prevents cross-role conflicts
Theming	CSS variables — full dark/light with anti-flash on load
Reusable components	15+ components: Button, Input, Select, Textarea, Badge, Card, Modal, Skeleton, Pagination, SearchBar, EmptyState

Route Map

Route	Page	Access
/	Job List	Public
/jobs/:id	Job Detail	Public

/login, /register	Authentication	Public
/dashboard	Accountant Dashboard	Accountant
/my-bids	My Bids	Accountant
/profile	Accountant Profile (4 tabs)	Accountant
/client/dashboard	Client Dashboard	Client
/client/jobs/new	Post Job	Client
/client/jobs/:id	View Bids + Accept/Reject	Client
/accountant/:id	Bidder Profile (access-gated)	Client
/admin + sub-routes	Full Admin Panel	Admin
/admin/audit-logs	Audit Log Viewer	Admin

7 SECURITY & BUSINESS LOGIC

JWT Authentication	24-hour expiry, bcrypt password hashing, tab-isolated sessionStorage, role-based middleware on all protected routes.
Duplicate Bid Prevention	Three-layer defence: UNIQUE constraint (job_id, accountant_id) at database level, checked at application service layer, and 'Already Submitted' state shown immediately in frontend via current_user_bid field in the job detail API response.
NDA Compliance	Per-job flag, DB-driven NDA text, acceptance logged with user_id + job_id + timestamp + IP. UNIQUE constraint prevents double-logging.
Job Closure Logic	Acceptance triggers: job closes, accepted bid → 'accepted' status, all other pending bids → 'rejected'. Deadline passes: auto-closes with reason 'deadline_passed'. Manual: 'cancelled_by_client' or 'admin_closed'.
Audit Trail	10 event types logged immutably: user.registered, user.login, profile.updated, job.created, job.updated, job.status_changed, bid.submitted, bid.accepted, bid.rejected, nda.accepted. Each record captures before/after JSONB, IP, user agent. Admins control their own logging with critical actions always captured.
Input Validation	Pydantic schemas validate all API inputs server-side. Inline JS validation on all frontend forms. Budget range CHECK constraint at database level.

8 ADMINISTRATION ROLE

A complete administrative role was built beyond the scope of the project requirements. This role represents the platform operator and has full visibility and control over all system entities.

Capability	Description
Dashboard Stats	Real-time breakdown of users by role, jobs by status, bids by status.
User Management	Full CRUD on all user accounts, with per-user JSONB restriction scoping (e.g. limit posting to specific cate
Content Management	DB-driven platform text: NDA templates, disclaimers, reminders — all editable without a deployment.
Document Verification	Admin reviews and verifies accountant-uploaded PDFs and CVs.
Audit Log Viewer	Immutable, filterable log of all platform events. Filter by category, user role, date range, and free-text searc
Audit Toggle	Admins can toggle logging of their own routine actions. Critical actions (user restrictions, content changes)
Reference Data Management	Full CRUD for categories, certifications, software skills, and countries — the domain vocabulary is admin-c
Job & Bid Oversight	Admins can view, edit, or close any job and manage any bid across all users.

9 WHAT WAS BUILT — FULL SYSTEM OVERVIEW

The following table maps each dimension of the system against what was implemented, providing a concise reference of the full scope delivered.

Dimension	What Was Implemented
Product Thinking	Architecture-first approach. Domain-specific modeling for accounting. NDA gates, structured bids, match scor
UI/UX	Clean dark/light theme, custom design system inspired by Upwork's UX patterns and adapted for the account
Frontend	React 18 + Vite, TanStack Query v5 for server state, React Context for auth, sessionStorage tab isolation, 4 r
Backend	FastAPI with async SQLAlchemy, Pydantic validation on every endpoint, 77 endpoints across 4 contexts, aut
Database	17 tables, domain-specific JSONB with GIN indexes, CHECK constraints, denormalized performance counter
Code Organisation	Feature-based folder structure, shared ui/ component library, services layer, schemas separated from routes,
Business Logic	Triple-layer duplicate bid prevention, NDA gate with DB log, match scoring engine, job closure with cascading
Deployment	Deployed on a personal server using Docker. The React frontend is built and served via a Docker image host
Attention to Detail	Company names on job cards dynamically resolved, bidder profile access-gated by bid relationship, expired c
Solo Build Scope	Full-stack solo delivery: database, API, frontend, auth, admin, i18n, theming, domain modeling — all designe

10 DELIVERED FEATURES & BEYOND

All required features and all stated bonus points were delivered. Additionally, 10+ features were built beyond any stated requirement, demonstrating depth of product thinking.

Bonus Points — All Delivered

Bonus Item	Implementation
Dashboard for submitted bids	Accountant Dashboard + Client Dashboard with live stats
Job status: Open / Closed	Full filtering, status badges, closed_reason tracking (4 reasons)
Pagination for job listing	Backend offset/limit + TanStack Query + custom Pagination component
Better filtering and sorting	10 filter parameters, 5 sort options — including domain-specific filters (jurisdiction, accounting standard)
Reusable UI components	15+ components in ui/ folder with consistent props API
API Resources / clean formatting	Standard {data, meta} envelope on all list endpoints
Deployment with live URL	Deployed on own server using Docker. React frontend served via Docker image registry. Backend + DB
README with setup instructions	Comprehensive README with env setup, seed data, API reference
Seeded demo data	10 jobs, 7 users, 10 countries, 24 categories, 8 certifications, 14 software skills, 9 content items

Extra Credit — Beyond Requirements

Extra Feature	Description
Admin Role + Full Panel	Complete admin dashboard: user management, CRUD for all entities, audit log viewer, content management
Client Role + Job Posting	Full client workflow: post, edit, close jobs, accept/reject bids, view bidder profiles
Hierarchical Categories	6 parent + 18 child accounting taxonomy with self-referencing schema
Certification Expiry Tracking	30-day warning, expired exclusion from match scores
Jurisdiction & Standards Matching	Cross-referenced in 5-dimension match engine
Software Skills Tracking	14 accounting software tools with profile tagging
NDA Gate per Job	DB-driven NDA text, acceptance audit trail, IP logging
Match Scoring Engine	5-dimension matching with visual percentage score
Structured Bid Proposals	10+ fields including milestones, engagement terms, jurisdiction confirmation
Audit Logging System	10 event types, immutable, filterable, admin-controllable
Dark/Light Theme	CSS variables, localStorage persistence, anti-flash on load
EN/AR Language Toggle	Full i18n with RTL layout, Cairo font for Arabic
Document Upload	PDF/DOC/PNG/JPG with admin verification workflow
User Restrictions (Admin)	JSONB permissions scoping per user account

Bidder Profile View (Client)	Access-gated: only viewable if accountant bid on that client's job

11 FUTURE ENHANCEMENTS & AUTOPILOT ROADMAP

The following represents the production evolution roadmap. These features were excluded from the initial build to maintain scope but were designed with future implementation in mind — the schema and API structure anticipate them.

Near-Term (Autopilot Stage)

Professional Messaging Channel

Upon bid acceptance, a dedicated, monitored messaging channel opens between the client and accountant. Messages are content-moderated to prevent sharing of off-platform contact details, illegal content, or scope violations. The channel persists for the engagement lifetime and is archived post-completion for audit purposes. WebSocket-based with full message history stored.

Idempotency Keys

POST endpoint middleware to prevent duplicate submissions from network retries. Critical for bid submission and job creation reliability.

Cloud File Storage (S3/Cloudinary)

Migrate uploaded documents from local storage. The file_url database field is already a URL string — zero schema changes required.

Automated Job Closure

Background task (Celery/APScheduler) auto-closes jobs when deadline passes, setting closed_reason = 'deadline_passed' without manual intervention.

Email Notifications

Transactional emails for bid confirmation, acceptance/rejection, and NDA acceptance via SendGrid or AWS SES.

Two-Factor Authentication

TOTP-based 2FA for all roles, admin-enforced for accountants handling sensitive financial data.

Rate Limiting

Slowapi middleware on auth endpoints (brute force prevention) and bid submission (spam prevention).

Company Verification

Domain email matching, business licence upload, admin review workflow, verified badge display.

CI/CD Pipeline

GitHub Actions workflow: run tests, build Docker images, push to registry, and auto-deploy to the server — extending the existing Docker-based deployment setup.

Payment & Escrow

Milestone-based payment releases via Stripe. Aligned with the structured bid milestone fields already captured in the bids schema.

Long-Term — Agentic AI Integration

If Accountant Hub is taken seriously as a platform product, there is a clear and compelling vertical for integrating agentic AI workflows across the core user journeys.

◆ **Accountant-Side AI Agent**

An AI agent that reviews a job posting, cross-references the accountant's profile, identifies gaps, and drafts a tailored structured bid — including proposed milestones, engagement terms, and jurisdiction confirmation. The accountant reviews and approves before submission.

- **Client-Side AI Agent**
  An agent that helps clients write precise job postings: asking clarifying questions about deliverables, jurisdiction, standards required, and expected outputs — then generating a well-structured posting that attracts the right bids.
- **Bid Evaluation Agent**
  When a client receives multiple bids, an AI summary agent ranks and explains bids based on match score, proposed price, timeline, and engagement terms — helping non-expert clients make informed hiring decisions.
- **Compliance Monitoring Agent**
  Monitors the engagement messaging channel for compliance signals: unreported scope expansion, off-platform contact attempts, regulatory concerns — flagging for admin review.
- **Platform Intelligence Layer**
  Aggregated analytics agent surfacing insights: which certifications are most in-demand by jurisdiction, underserved accounting niches, pricing benchmarks by service type — turning platform data into actionable market intelligence for both accountants and clients.

The existing architecture — structured bids, match scoring, audit trails, JSONB profile fields, and well-defined API contracts — is already a strong foundation for these agentic layers. No schema redesign would be required to add AI-powered features on top of the current system.

12 APPENDIX: TEST CREDENTIALS & SETUP

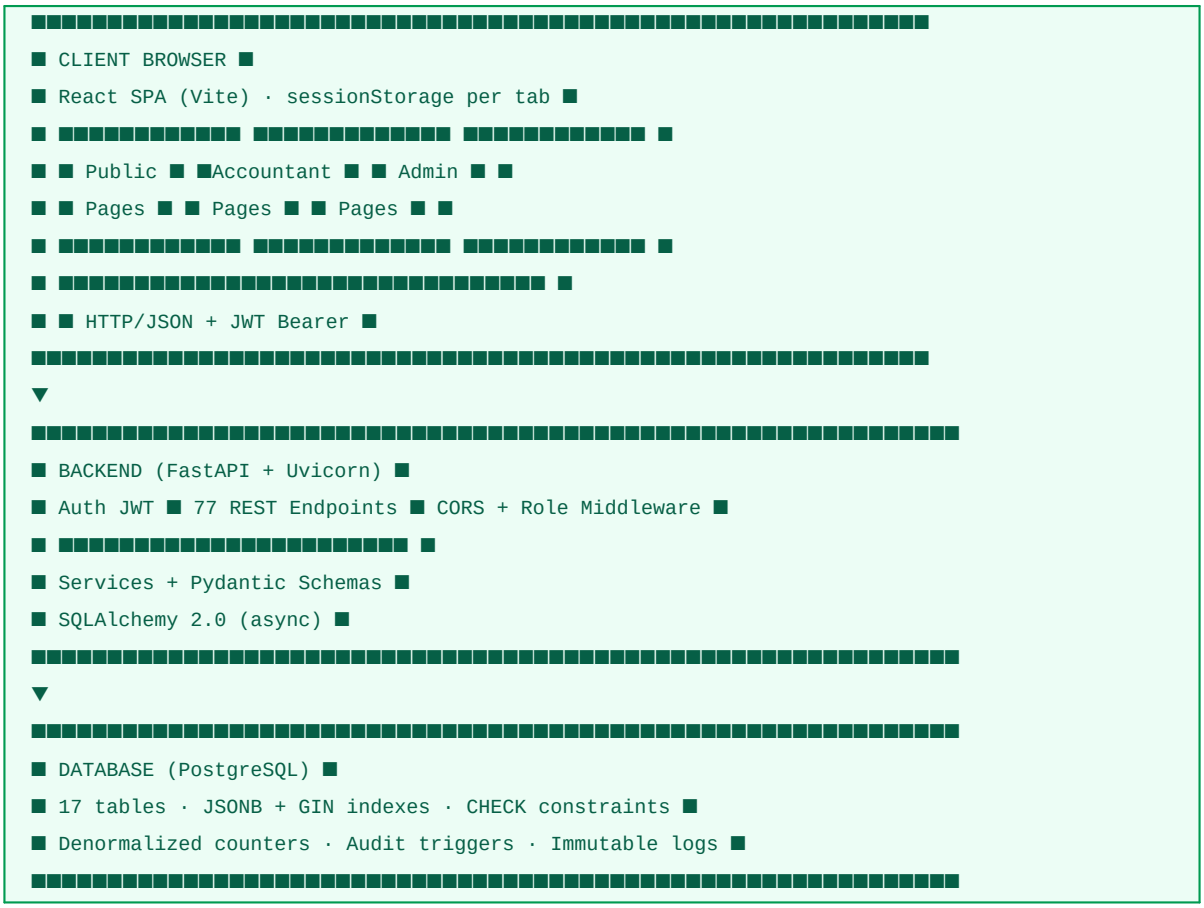
Test Credentials

Role	Email	Password
Admin	admin@accounthub.com	Admin123!
Accountant	ahmed@example.com	Ahmed123!
Accountant	sarah@example.com	Sarah123!
Accountant	john@example.com	John1234!
Client	deloitte@example.com	Deloitte1!
Client	emirates@example.com	Emirates1!
Client	techventures@example.com	TechVent1!

Seeded Demo Data

Entity	Count	Notes
Jobs	10	Across all categories, mixed open/closed
Users	7	1 admin, 3 accountants, 3 clients
Countries	10	With currency codes and phone validation
Categories	24	6 parent + 18 child accounting taxonomy
Certifications	8	CPA, CMA, ACCA, EA, CFA, CIA, DipIFRS, SOCPA
Software Skills	14	QuickBooks, Xero, SAP, Oracle NetSuite + 10 others
Content Items	9	NDA text, disclaimers, platform reminders

System Architecture Summary



Mohamed Hossam Hassan mohamed.hhossam@alum.bue.edu.eg +201028992674

End of Architecture & Technical Design Report